ARIITK HACKATHON 2026

Hey Y25s ! Hope you are enjoying your time on the campus . These are problem statements of the Hackathon 2026. These are divided into three levels. The submission guidelines of each level is mentioned clearly. Please follow them correctly. Any doubts can be raised on GitHub in Discussions in the following repository. You can find the problem statements and required starter kits of the problem statements in this repo

```
https://github.com/AerialRobotics-IITK/ARIITK_Hackathon
```

Level 1 — Operation SKYE-X: Search and Pursuit

> Team Aerial Robotics | Y25 Recruitment Hackathon

---

1. Context

A rogue adversarial drone — designation **SKYE-X** — has been detected inside a classified industrial facility. Intel suggests it is equipped with onboard evasion algorithms and is actively trying to break line-of-sight, hide behind obstacles, and escape your tracking radius.

You are the operator of an autonomous interceptor quadcopter. You have no direct control — your drone acts on the instructions of the AI controller you write. Your sensors are limited. The map is mostly hidden behind Fog of War. The target knows you are coming.

Your mission is to navigate the facility, locate SKYE-X, and shadow it persistently within a **70-pixel tracking radius** for as long as possible before the mission clock runs out.

SKYE-X will not make this easy.

---

2. The Enemy — SKYE-X

SKYE-X is not a passive target. It has its own onboard AI with three distinct behavioural layers that activate depending on how close you are:

Behaviour	Trigger	What SKYE-X Does
Gaussian Random Walk	Always active	Moves with smooth, unpredictable momentum — never fully still, never fully predictable
Seek Cover	You within `80 px`	Identifies the nearest obstacle and steers toward it to break your line-of-sight. **This is its most dangerous capability.**
Direct Escape	You within `80 px`, no cover nearby	Flees directly away from you at full speed
Wall Avoidance	Near arena edges	Soft repulsion keeps it from cornering itself — it will not trap itself for you

> The cover-seeking behaviour is what separates good controllers from great ones. SKYE-X actively places rectangular obstacles between itself and you, causing `target_visible` to drop to `False` and forcing you back into blind exploration. If your pursuit logic cannot recover from a lost target, your score will stall.

---

3. The Environment

Arena

Parameter	Value
Width × Height	`1280 × 900` pixels
Obstacles	`22` randomized rectangles (crates, pillars, barriers)
Player Spawn	`(80, 80)`
Target Spawn	`(WIDTH - 150, HEIGHT - 150)` ≈ `(1130, 750)`
Mission Duration	`3000` timesteps (default — configurable via `config.py`)
Frame Rate	`60 FPS`

Drone Physics

Parameter	Value
Max Player Speed	`3.5` px/timestep
Player Collision Radius	`10` px
Max Target Speed (Normal)	`3.5` px/timestep

> **Note:** Speed is enforced — if your output velocity vector exceeds `MAX_PLAYER_SPEED`, it is automatically clamped to that magnitude.

---

4. Sensor Suite — What You Can See

Your only window into the world is the `get_sensor_data()` return dictionary:

```
sensors = {
    "player_x":        float,   # Your X coordinate
    "player_y":        float,   # Your Y coordinate
    "lidar_distances": list,    # 36 floats — one per ray (every 10°)
    "target_visible":  bool,    # True if target is within VISIBILITY_RADI
US (150px)
    "target_pos":      tuple    # (tx, ty) if visible, else None
}
```

LiDAR System

- **36 rays**, equally spaced every **10°** (full 360° sweep)
- Each reading is the distance to the nearest obstacle/wall in that direction
- Maximum range: **150 px** (Visibility Radius)
- Beyond that range: the map is hidden behind **Fog of War**

Visibility

- You can only detect the target if it is within **150 px** of your position
- `target_visible` becomes `True` the moment the target enters this radius
- Once discovered (`target_discovered = True`), scoring begins

---

5. Collision Rules

Your drone must avoid:

1. **Arena boundaries** — flying outside `[0, WIDTH] × [0, HEIGHT]` instantly crashes the drone.
2. **Rectangular obstacles** — collision is detected using circle-AABB: if your drone's center comes within **10 px** (DRONE_RADIUS) of any obstacle edge, it crashes.

A crash immediately ends the mission with your current score — no partial credit.

---

6. Objectives & Scoring

Score Accumulation

- **+1 point per timestep** that the following conditions are **both** met:

1. The target has been discovered at least once (`target_discovered == True`)
2. Your drone is within **70 px** (TRACKING_RADIUS) of the target's current position

Mission Termination

The mission ends when **any** of these occur:

- `timesteps >= MAX_TIMESTEPS` (default `3000`, configurable in `config.py`)
- Player drone crashes into a wall or obstacle

What Determines a Good Score?

- **Fast exploration**: Find the target quickly to start accumulating points early
- **Persistent pursuit**: Stay within 70 px despite the target actively fleeing
- **Obstacle awareness**: Cut through the maze efficiently without crashing

---

7. Your Task

Open `skye_controller.py` and implement the `compute_velocity(sensors)` function.

```
def compute_velocity(sensors) -> (float, float):
    """
    Input : sensors dict (see Section 3)
    Output: (vx, vy) — your velocity command for this timestep
            ||(vx, vy)|| will be clamped to MAX_PLAYER_SPEED = 3.5
    """
    ...
```

A starter implementation with a **dual-state template** is already provided:

- **State 1 (Exploration):** When `target_visible == False` — explore the map
- **State 2 (Pursuit):** When `target_visible == True` — chase and track the target

You are free to use any approach: hand-tuned heuristics, potential fields, A* path planning, Reinforcement Learning, or anything else.

---

8. Deliverables

Teams must submit a single GitHub repository containing all of the following:

#	File	Description
1	`skye_controller.py`	Your implementation of `compute_velocity(sensors)`. This is the only file you modify. Must run without errors on the unmodified `skye_env.py` and `config.py` provided.
2	`score_proof.png`	A screenshot of your final simulation run showing the score on screen. Taken at mission end (timestep 3000 or crash).

---

9. Constraints

Rule	Detail
Modify	`skye_controller.py` (your main solution)
May modify	`config.py` — you may adjust parameters like `MAX_TIMESTEPS` for tuning/testing
Do not modify	`skye_env.py`
Do not	Hard-code target coordinates or exploit rendering internals
Allowed	Any Python library (`numpy`, `scipy`, etc.)

---

10. Scoring Summary

Outcome	Score
Crashed immediately	`0`
Found target but failed to track	Low (< 50)
Found & tracked for most of mission	Medium (200–800)
Perfect pursuit + survival	High (> 1000)

Level 2 — Operation Touchdown: Precision Landing

> Team Aerial Robotics | Y25 Recruitment Hackathon

---

1. Context

A drone is descending. You cannot stop it. The landing platform is oscillating sinusoidally below you — it never stops moving. Your only sensor is a downward-facing camera. You must steer the drone laterally so it touches down on the platform centre. No GPS. No position sensor. Everything inferred from pixels.

---

2. Environment

Simulation

Parameter	Value
Sim Duration	`35 seconds` — fixed descent, cannot be paused
Frame Rate	`30 FPS` (dt = 0.033 s)
Success Radius	Final error ≤ `0.05 m` from platform centre
Drone Start	`(80 px, 180 px)` — top-left, far from platform
Altitude	`10.0 m → 0.0 m`, auto-descending (uncontrollable)
Max Speed	`5.0 m/s` lateral (simulator enforced)

Landing Platform

Parameter	Value
Size	`1 m × 1 m`
X Motion	`centre_x + 2.0 × sin(0.9 t) m` — SHM, ±2 m amplitude
Y Motion	`centre_y + 0.6 × sin(0.45 t) m` — slow sinusoidal drift

---

3. Camera

Each frame `step_env()` returns a flat list of **10 000 grayscale integers** (100 × 100 px, row-major).

Field of view: `fov_m = 0.30 × altitude` metres. Use this to convert pixel offsets to real-world metres.

Region	Gray Value	Notes
Grass / background	~45–90	Dark green
Platform surface	~200–230	Bright rectangle — the landing pad
Inner square	~0–20	Near-black square at platform centre

---

4. Your Task

Edit `solver.py` only. Implement the six TODOs:

TODO	Where	What
1	`SEARCH_SPEED`	Drone speed (m/s) during search.
2	`KP/KI/KD` constants	PID gains for X and Y axes.
3	`detect_platform()`	Find platform in pixel array. Return `(found, cx_norm, cy_norm)`.
4	`PID.update()`	Implement P + I + D with anti-windup. Return clamped velocity.
5	`search_velocity()`	Design a search pattern to sweep the arena.
6	`main()` — PID block	Convert `cx_norm / cy_norm` → metres via `sim.fov_m`. Feed into PID.

Phase 1 — Search

The drone starts top-left; the platform is not visible. Implement `search_velocity()` to sweep the arena until the platform enters the camera FOV.

Phase 2 — Track & Land

Once detected, use PID to keep the drone centred over the moving platform throughout descent.

```
err_x_m = cx_norm * (sim.fov_m / 2)
err_y_m = cy_norm * (sim.fov_m / 2)
```

```
vx = pid_x.update(err_x_m, dt)
vy = pid_y.update(err_y_m, dt)
```

5. Scoring

Outcome	Points
Final error ≤ `0.20 m`	10
Final error ≤ `0.10 m`	20
Final error ≤ `0.05 m` **(SUCCESS)**	50
Bonus: error ≤ `0.02 m`	+20

6. Rules

Allowed

- Edit `solver.py` freely — add functions, tune constants, import libraries.
- Use any pip-installable package (NumPy, OpenCV, SciPy, etc.).

Not Allowed

- Modify `simulator_level2.py`.
- Access `sim.plat_x`, `sim.plat_y`, or any internal simulator variable.
- Hardcode the platform position or trajectory.

Deliverables

Submit **`solver.py`** only. Must run with the original unmodified `simulator_level2.py` in the same directory:

```
python solver.py
```

Level 3 — Project MIRAGE

> Team Aerial Robotics | Y25 Recruitment Hackathon

1. Background

Modern autonomous drone swarms rely on passive ground markers such as ArUco codes and QR codes for navigation, zone identification, and precision landing. However, these markers have a critical security weakness: they broadcast their information to every camera at every altitude. Any friendly drone — or an enemy drone — that flies overhead can read them.

We need a smarter marker. One that tells different information depending on how close the drone is.

The human visual system and any camera acts as a spatial frequency filter. At high altitude, a camera captures only **low spatial frequencies** (smooth gradients, general shapes, broad colours). At low altitude, it captures **high spatial frequencies** (sharp edges, fine binary patterns, precise detail). By engineering an image that separates information across these two frequency bands, we can create a single, flat, passive marker that reveals different content at different altitudes.

---

2. The Challenge

Design and implement a complete algorithmic pipeline that produces a single image — called a **Hybrid Marker** — with the following behaviour:

Drone Altitude	What the Drone Sees	ArUco Detection
≥ 30 metres (High)	Decoy image — a large, bold H helipad symbol	Must **FAIL**
8 – 20 metres (Transition)	Partial overlap of both images	Should **FAIL**
≤ 5 metres (Low)	Secret ArUco marker with the announced ID_X	Must **SUCCEED**

The marker must be a **single flat** `.png` **file**. No layers, no metadata, no active components.

---

3. How Altitude is Simulated (No Drone Required)

Since no physical drone, ROS, or Gazebo is available, altitude is simulated mathematically using the **Pinhole Camera Model**. The formula below calculates exactly how many pixels the printed marker occupies on the drone's camera sensor at a given real-world altitude:

```
Pixels on Sensor = (f × S_real × Resolution) / (Z × Sensor_width)
```

Fixed Hardware Constants (Raspberry Pi Camera V2)

Parameter	Value
Focal length (f)	`3.04 mm`
Sensor width	`3.68 mm`
Resolution	`1920 pixels` (width)
Physical marker size (S_real)	`1.0 metre`

Altitude vs Sensor Pixels

Altitude (Z)	Pixels on Sensor	Visual Result
2 m	~793 px	Full ArUco detail visible
5 m	~317 px	ArUco still readable
15 m	~105 px	Transition — both blending

Altitude (Z)	Pixels on Sensor	Visual Result
30 m	~52 px	Only decoy visible
100 m	~15 px	Fully blurred, only color/shape

> **Hint:** To simulate altitude Z in software, teams can shrink their 512×512 hybrid image down to the calculated pixel size using `cv2.resize()`, then optionally apply a mild Gaussian blur to simulate atmospheric haze. This single operation sufficiently replicates what a real drone camera would see at that height.

---

4. Technical Constraints

Rule	Detail
Output image	Must be exactly `512 × 512` pixels, grayscale (single channel)
ArUco dictionary	Must be `DICT_4X4_50`
Language	Python 3 with OpenCV (`opencv-contrib-python`) and NumPy
ArUco ID	Announced at hackathon start — must be accepted as a command-line argument. Hardcoding any ID = **immediate disqualification**

---

5. Deliverables

Teams must submit a single GitHub repository containing all of the following:

File	Description
`generator.py`	Takes `far_target_decoy.png` and the announced ArUco marker from the `aruco_markers/` folder as inputs. Outputs `hybrid_marker.png`. Teams must tune `σ_low` and `σ_high` using the Pinhole Camera Model math. **Usage:** `python generator.py decoy.png aruco_markers/aruco_id_X.png X output.png`
`hybrid_marker.png`	The final 512×512 grayscale hybrid marker. This is the **primary judged artifact**.
`viewer.py`	**Build from scratch — NOT provided in starter kit.** Takes image path, altitude in metres, and ArUco ID as command-line arguments. Simulates what the drone sees, saves the output image, and prints whether ArUco is detected. **Usage:** `python viewer.py hybrid_marker.png 30 X`
`visual_check.py`	**Build from scratch — NOT provided in starter kit.** Measures the ghosting score of the hybrid marker. Must print a numeric ghosting score. Submitted score must be below 5. **Usage:** `python visual_check.py hybrid_marker.png far_target_decoy.png`
`drone_view_2.0m.png`	Saved output of `viewer.py` at 2 metres — **mandatory visual proof**.
`drone_view_30.0m.png`	Saved output of `viewer.py` at 30 metres — **mandatory visual proof**.
`report.pdf`	Maximum 2 pages. Must include: the σ values chosen, the mathematical reasoning behind them, and the GSD calculation proving the transition altitude.

`viewer.py` — Usage & Expected Output

> `viewer.py` and `visual_check.py` are **NOT provided** in the starter kit. Teams must build both from scratch. Submitting a blank file or a file that does not run will result in **zero marks** for those criteria.

`viewer.py` must accept three command-line arguments — image path, altitude in metres, and ArUco ID:

```
python viewer.py hybrid_marker.png 30 X
python viewer.py hybrid_marker.png 2 X
```

Expected terminal output at 30 metres:

```
Drone at 30.0m → Marker occupies 52×52 pixels on sensor
Atmospheric blur: Applied
Drone sees: DECOY IMAGE dominant
ArUco ID_X NOT DETECTED → Continue cruising
Image saved → drone_view_30.0m.png
```

Expected terminal output at 2 metres:

```
Drone at 2.0m → Marker occupies 793×793 pixels on sensor
Atmospheric blur: None
Drone sees: ARUCO MARKER dominant
ArUco ID_X DETECTED → Precision landing zone identified
Image saved → drone_view_2.0m.png
```

Saved image format: The saved `drone_view` PNG must show a pixelated drone POV view at that height. It must include a top bar showing altitude and sensor resolution, the drone view in the centre, and a coloured bottom banner — green if ArUco detected, red if not.

> **Important:** Judges will call `viewer.py` live during evaluation with random altitude values. Your script must handle any positive altitude correctly.

---

6. Ghosting — What It Is & Why It Costs Points

Ghosting is the most common failure mode. It occurs when the ArUco marker's black grid pattern is faintly visible in the high-altitude view, meaning the decoy has been contaminated by the near-target's low-frequency content.

To avoid ghosting:

- **Normalise the high-pass signal** — subtract its mean before adding it to the decoy so it carries no DC offset
- **Increase the sigma gap** — `σ_low` must be at least **3×** larger than `σ_high`

Teams must measure their own ghosting score using their `visual_check.py` and include the score in their submission. Aim for a ghosting score **below 5**. Above 15 will visibly fail the judges' eye test and lose all visual quality marks.

---

7. Judging & Evaluation Criteria

Criterion	Weightage	How It Is Tested
ArUco ID_X detected at ≤ 5m	**40%**	`detector.py` shrinks image to 5m pixel size and runs ArUco detection
ArUco ID_X hidden at ≥ 30m	**30%**	`detector.py` shrinks image to 30m pixel size and runs ArUco detection
Visual quality — no ghosting on decoy	**15%**	Judging script run at 30m; judges visually inspect the window for ArUco grid bleed-through
Math justification in `report.pdf`	**15%**	Judges verify σ values are backed by GSD/Nyquist calculations, not trial and error

---

8. Starter Kit

```
Mirage-Starter-Kit/
│
├── far_target_decoy.png        ← one decoy image
│
├── aruco_markers/              ← all 50 markers pre-generated
│   ├── aruco_id_00.png
│   ├── aruco_id_01.png
│   ├── aruco_id_02.png
│   ├── ...
│   └── aruco_id_49.png
│
├── generator.py               ← badly tuned template
├── simulator.py               ← altitude tester
└── README.md
```

Bonus Level — Operation FREEFALL: Autonomous Precision Landing

> Team Aerial Robotics IITK| Y25 Recruitment Hackathon

---

1. Context

Your autonomous drone must execute a **precision landing sequence** inside a high-fidelity 3D physics simulator.

Beneath your drone are landing pads marked with ArUco markers. Your objective is to design a closed-loop flight controller that uses Computer Vision to detect the marker in real-time (capturing the simulator screen using a library such as PyAutoGUI for screenshot acquisition), calculates the positional

error, and sends flight commands to land the drone precisely on the target — without ever hardcoding coordinates.

You will not write code inside the simulator. Instead, your Python script (`auto_land.py`) acts as an **external flight computer**, communicating with the simulator over the network.

---

2. The Mission Phases

This level is divided into two escalating challenges:

Phase 1 — The Static Dropzone

Parameter	Detail
Target	A stationary landing pad equipped with an ArUco marker
Objective	Take off, visually acquire the marker, align X and Z axes, and steadily decrease altitude (Y-axis) until touchdown
Success Criteria	The drone rests completely inside the marker's boundary for at least **3 seconds** without tipping over

Phase 2 — The Moving Platform (Dynamic Dropzone)

Parameter	Detail
Target	An ArUco marker mounted on an Autonomous Ground Vehicle (AGV) moving unpredictably across the ground
Objective	Intercept the moving target, match its velocity, maintain vertical alignment, and execute a dynamic landing
Success Criteria	Touchdown and stable rest on the moving platform

---

3. Simulator Communication Protocol (The API)

Your Python script communicates with the simulator via **network sockets** and **HTTP requests**. The simulator binaries are too large for GitHub — download them from the link below.

> **Download Simulator:** Google Drive — Bonus Level Simulator
>
> | File | Platform |
> |---|---|
> | `drone (1).exe` | Windows |
> | `drone.x86_64` | Linux |
> | `drone.zip` | MacOS |

A. Telemetry Stream (Simulator → Your Script)

The simulator broadcasts real-time physics data over a **TCP socket** at `127.0.0.1` on port **8081**.

Manual test (Netcat):

```
nc 127.0.0.1 8081
```

Data Format: Continuous JSON strings. Note that the **Y-axis represents altitude**.

```
{
  "name": "Drone",
  "position": [0.0, 1.35060441493988, 0.0],
  "rotation": [0.0, 0.0, 0.0],
  "velocity": [0.0, 0.232620656490326, 0.0]
}
```

Field	Description
`position`	`[X, Y, Z]` — Y is altitude
`rotation`	`[pitch, yaw, roll]` in degrees
`velocity`	`[vX, vY, vZ]` — vY is vertical velocity

B. Flight Control (Your Script → Simulator)

Send flight commands via **HTTP POST** to `http://localhost:8080`.

Manual test (cURL):

```
curl localhost:8080 --json '{ "pitch": -1.0 }'
```

Command Dictionary: Values typically range from `-1.0` to `1.0`.

Command	Effect
`"pitch"`	Forward / backward tilt
`"roll"`	Left / right tilt
`"yaw_rate"`	Left / right rotation
`"throttle"`	Vertical thrust

You can send any combination of these in a single JSON payload:

```
{ "pitch": -0.3, "roll": 0.1, "throttle": 0.8 }
```

---

4. Your Task — The Control Loop

Write a Python script `auto_land.py` that implements a continuous feedback loop at approximately **30–60 Hz**:

```
┌──────────────────────────────────────────────────────────┐
│                      CONTROL LOOP                        │
│                                                          │
│  1. READ     ← Telemetry from port 8081                  │
│              ← Camera frame by capturing window          │
│                                                          │
│  2. PROCESS  → Detect ArUco marker (OpenCV)              │
│              → Calculate pixel error from frame center   │
```

```
│                                                          │
│  3. COMPUTE  → PID Controller converts visual error      │
│                into smooth flight commands               │
│                                                          │
│  4. ACT      → Send pitch, roll, throttle, yaw to 8080   │
│                                                          │
│  └── Repeat ──────────────────────────────────────────┘ │
└──────────────────────────────────────────────────────────┘
```

5. Constraints & Guidelines

Rule	Detail
Allowed Libraries	`requests`, `socket`, `opencv-python` (cv2), `numpy`, `simple_pid`, `pyautogui`
No Hardcoding	You **cannot** hardcode the target's physical coordinates. The drone **MUST** use the camera feed and ArUco detection to find the landing zone.
Out of Bounds	Flying the drone outside the simulated arena walls results in immediate failure.
ArUco Dictionary	Must use `DICT_4X4_50`

---

6. Deliverables

Teams must submit a single GitHub repository containing:

#	File	Description
1	`auto_land.py`	Your main control loop script. Must connect to the simulator, detect ArUco markers, and land the drone autonomously.
2	`landing_video.mp4`	A screen recording of a successful landing (Phase 1 minimum, Phase 2 for full marks).
3	`report.pdf`	Maximum 2 pages. Must include: your PID tuning approach, your ArUco detection pipeline, how you handle Phase 2's moving target, and a block diagram of your control loop.

`report.pdf` Must Answer These Questions

- How does your controller detect and lock onto the ArUco marker?
- How do you convert pixel error to physical flight commands?
- What PID gains did you use and how did you tune them?
- How does your system handle the marker going out of the camera frame?
- How does your approach change for the moving platform in Phase 2?

---

7. Scoring

Metric	Weightage
Phase 1 Completion (static landing)	**30%**
Phase 2 Completion (moving platform)	**50%**
Landing Smoothness	Up to **20%** bonus (touchdown velocity closer to 0.0 m/s → higher score)
Time to Land	Faster landings break ties

---

8. Running the Simulator

Step 1: Download the simulator from Google Drive.

Step 2: Run the simulator for your platform:

Windows

```
.\\"drone (1).exe"
```

or double click on the exe

Linux

```
chmod +x drone.x86_64
./drone.x86_64
```

Or give executable permission in properties and double click

Mac OS

Run from the zip

Step 3: In a separate terminal, run your controller:

```
python auto_land.py
```

> The simulator window will show the drone responding to your commands in real-time.